Unidad 6: Sincronización y Transacciones Distribuidas

Basado en Coulouris et al., *Distributed Systems: Concepts and Design* (Cap. 14, 15, 16 y 17)

6.1 Sincronización

¿Qué es la sincronización en SSDD?

En un sistema distribuido, cada nodo corre con su propio reloj de hardware a su propia velocidad. No existe un reloj global compartido, y los mensajes tardan tiempos impredecibles en llegar. Esto hace que coordinar el orden de los eventos entre procesos sea un problema no trivial y central en el diseño de sistemas distribuidos.

La sincronización aborda la pregunta: **¿cómo hacen múltiples procesos distribuidos para acordar en qué orden ocurrieron las cosas, y cómo acceden a recursos compartidos sin pisarse?**

Tipos de Sincronización

La sincronización se divide en dos grandes niveles:

De bajo nivel (mecanismos de coordinación):

- Paso de mensajes como base de la comunicación entre procesos
- Exclusión mutua distribuida
- Relojes lógicos (ordenamiento de eventos)

De alto nivel (abstracciones para aplicaciones):

- Consistencia de datos (replicación, coherencia)
- Transacciones distribuidas
- Modelos de sistema (estados globales, snapshots)

Relojes en los Sistemas Distribuidos

El problema del hardware

Cada computadora tiene un **dispositivo reloj** basado en un cristal de cuarzo que oscila a una frecuencia definida según su corte y tensión. El problema es que dos cristales distintos nunca oscilan exactamente igual: con el tiempo sus lecturas divergen. A esta diferencia acumulada se la llama **deriva del reloj** (*clock drift*).

En una única máquina esto no importa porque todos los procesos comparten el mismo reloj. Pero en un sistema distribuido con múltiples nodos, la deriva puede producir fallas severas en aplicaciones que dependen del tiempo.

Ejemplo clásico (el problema del banco): una cuenta bancaria está distribuida en dos sucursales. El balance se determina a las 3:00 PM. Sucursal A transfiere $100 a las 3:01 (hora de A), pero el mensaje llega a Sucursal B a las 2:59 (hora de B). Resultado: el dinero se cuenta dos veces porque, desde el punto de vista de B, el mensaje llegó *antes* del corte. El desajuste de relojes generó una inconsistencia real.

Otro ejemplo: sistema de compilación distribuido en dos máquinas A y B. Si el reloj de B está atrasado, un archivo modificado en A a las 09:01 puede parecer más antiguo que el .o generado antes de esa modificación en B (cuyo reloj marca 09:00), y el sistema de make no recompila aunque debería.

Relojes físicos vs. relojes lógicos

Existen dos enfoques según la necesidad:

- **Relojes físicos**: necesarios cuando la aplicación requiere que los relojes no se desvíen del tiempo real (ej. logs de auditoría, sistemas financieros, certificados X.509). Se sincronizan contra fuentes externas (GPS, NTP).
- **Relojes lógicos**: suficientes cuando lo que importa no es la hora en sí, sino el **orden relativo** en que sucedieron los eventos. Es el enfoque de Lamport.

Sincronización de Relojes Físicos (Coulouris, Cap. 14.3)

Algoritmo de Cristian

Cristian (1989) propuso un algoritmo para sincronizar el reloj de un cliente contra un servidor de tiempo confiable:

1. El cliente registra el momento de envío `T0` y envía una solicitud de tiempo al servidor.
2. El servidor responde con su tiempo actual `Tserver`.
3. El cliente registra el momento de recepción `T1`.
4. El cliente estima el tiempo de transmisión de ida como `(T1 - T0) / 2` (asumiendo simetría).
5. El cliente ajusta su reloj a: `Tserver + (T1 - T0) / 2`.

Problema crítico: no se puede retroceder el reloj abruptamente. Si el reloj del cliente está adelantado, retrocederlo rompería la causalidad (un evento podría tener un timestamp anterior a otro que ocurrió antes). La solución es **ralentizar gradualmente el reloj** local hasta que el tiempo real lo alcance.

Limitación: si el servidor de tiempo falla, la sincronización es imposible. Cristian sugirió usar un grupo de servidores sincronizados; el cliente envía multicast y usa la primera respuesta.

Algoritmo de Berkeley

Gusella y Zatti (1989) desarrollaron un algoritmo de **sincronización interna** (sin referencia externa) para Berkeley UNIX:

1. Un proceso **master** es elegido como coordinador.
2. El master **sondea periódicamente** a todos los slaves, solicitando sus tiempos actuales.
3. El master estima el tiempo local de cada slave (usando round-trip time, similar a Cristian).
4. El master calcula el **promedio fault-tolerant**: descarta lecturas que difieran más de un umbral máximo (para eliminar relojes defectuosos).
5. El master envía a cada slave el **ajuste necesario** (delta positivo o negativo), no el tiempo absoluto (para minimizar la incertidumbre por transmisión).

Ventaja sobre Cristian: no requiere una fuente de tiempo externa. Sincroniza todos los nodos entre sí. Tolera relojes defectuosos.

Resultado experimental: Gusella y Zatti sincronizaron 15 computadoras con una precisión de 20–25 ms. La deriva máxima era $< 2\times10^{-5}$, con un round-trip máximo de 10 ms.

Si el master falla, se puede elegir un nuevo master usando algoritmos de elección (ver sección 6.1.6).

Network Time Protocol (NTP)

El **NTP** (Mills, 1995) es el estándar para sincronización de relojes en Internet. Sus objetivos de diseño son:

- Proveer sincronización a UTC con alta precisión en redes con grandes y variables latencias.
- Ser robusto ante pérdidas de conectividad (servidores redundantes, rutas redundantes).
- Escalar a millones de clientes.
- Proteger contra interferencia maliciosa (autenticación de fuentes).

Arquitectura de NTP — jerarquía de estratos (strata):

Stratum 0: Fuentes primarias (relojes atómicos, GPS)

Stratum 1: Servidores conectados directamente a Stratum 0

Stratum 2: Servidores sincronizados con Stratum 1

Stratum 3: ...

...

Stratum N: Cliente final

Cada nivel agrega una pequeña incertidumbre. En LANs bien configuradas se alcanza precisión de microsegundos; en Internet, típicamente < 10 ms.

Modos de operación de NTP:

- **Multicast**: un servidor NTP hace multicast en la LAN; los clientes escuchan y ajustan. Latencia asumida constante. Menor precisión.
- **Procedure-call (Cristian-like)**: el cliente contacta activamente al servidor, mide el round-trip y estima el offset. Mayor precisión.
- **Simétrico (entre peers)**: dos servidores NTP se sincronizan mutuamente, intercambiando paquetes bidireccionales. La mayor precisión, usada entre servidores de infraestructura.

La Relación "Ocurre Antes" (Happens-Before)

Lamport definió la relación **"ocurre antes"** (→) para capturar causalidad entre eventos sin necesidad de un reloj global:

La relación $a \rightarrow b$ se da si:

1. a y b ocurren en el **mismo proceso** y a se produce antes que b.

2. `a` es el evento de **envío** de un mensaje y `b` es el evento de **recepción** de ese mismo mensaje en otro proceso.
3. **Transitividad**: si `a → b` y `b → c`, entonces `a → c`.

Si dos eventos `x` e `y` están en procesos diferentes que no intercambian mensajes, entonces ni `x → y` ni `y → x` son verdaderos. En ese caso se dice que `x` e `y` son **eventos concurrentes** (`x // y`).

El Reloj Lógico de Lamport

Lamport propone asociar a cada evento `a` una etiqueta temporal `C(a)` tal que: si `a → b`, entonces `C(a) < C(b)`.

Algoritmo

Cada proceso `Pi` mantiene un contador `Ci`:

1. `Ci` se inicializa en 0.
2. Antes de ejecutar cualquier evento en `Pi`: `Ci := Ci + 1`.
3. Cuando `Pi` envía un mensaje, adjunta su valor actual `t = Ci`.
4. Cuando `Pi` recibe un mensaje con valor `t`, actualiza: `Ci := max(Ci, t) + 1` antes de etiquetar el evento de recepción.

Ejemplo:

- P1: sucede `a` → C1(a) := 1. Sucede `b` → C1(b) := 2. Envía m1 con t=2.
- P2: empieza en 0. Recibe m1 (evento `c`) → C2(c) := max(0,2)+1 = 3. Sucede `d` → C2(d) := 4.

Limitación: si `C(a) < C(b)`, NO necesariamente `a → b`. Puede ser que `a` y `b` sean concurrentes.

Relojes Lógicos Totalmente Ordenados

Para obtener un **orden total** (desempate entre eventos concurrentes), se extiende `C(a)` añadiendo el identificador del proceso como decimal:

Si `a` ocurre en el proceso `Pi`, se define: `C'(a) = C(a).i`

Esto garantiza que cualquier par de eventos tiene un orden definido y único.

Relojes Vectoriales

Los relojes de Lamport no permiten determinar si dos eventos son concurrentes. Los **relojes vectoriales** solucionan esto.

Cada proceso `Pi` mantiene un **vector** `Vi` de N entradas:

- `Vi[i]`: reloj lógico local de `Pi`.

- `Vi[j]` (j ≠ i): el último valor conocido del reloj de `Pj` por `Pi`.

Algoritmo de gestión del vector

- **Inicialización**: `Vi[j] = 0` para todo `i, j`.
- **Antes de un evento** en `Pi`: `Vi[i] := Vi[i] + 1`.
- **Al enviar**: `Pi` adjunta su vector `Vm = Vi`.
- **Al recibir** con vector `Vm`: `Vi[k] := max(Vi[k], Vm[k])` para cada `k`; luego `Vi[i] := Vi[i] + 1`.

Comprobación de causalidad

- `a → b` si y solo si `Va ≤ Vb` (componente a componente) Y existe al menos una componente estrictamente menor.
- Si no se cumple en ninguna dirección: `a // b` (concurrentes).

Estados Globales y Snapshots (Coulouris, Cap. 14.5)

¿Por qué necesitamos el estado global?

Hay tareas para las que necesitamos conocer el estado global del sistema:

- **Detección de garbage distribuido**: objetos que ya no son referenciados por ningún proceso.
- **Detección de deadlock**: condiciones de espera cíclica en grafos de espera (*wait-for graphs*).
- **Detección de terminación**: saber si un algoritmo distribuido terminó.
- **Checkpoint y recuperación**: guardar un estado consistente para poder recuperar el sistema ante fallos.

Es vital tener en cuenta el estado de **todos los procesos Y de todos los canales de comunicación** (mensajes en tránsito).

Concepto de Corte (Cut)

Un **corte** `C` de la ejecución en el sistema distribuido es el conjunto de prefijos de las historias de los procesos:

C = H1^c1 ∪ H2^c2 ∪ ... ∪ HN^cN

donde `Hi^ci` contiene los eventos procesados por `Pi` incluidos en el corte.

Corte consistente: un corte es consistente si para cualquier evento `ej ∈ C` y cualquier evento `ei` tal que `ei → ej`, se tiene que `ei ∈ C`. En otras palabras: si la recepción de un mensaje está en el corte, el envío de dicho mensaje también debe estar en el corte (no puede haber mensajes que "viajan hacia el pasado").

Corte inconsistente: existe un par de eventos `ej ∈ C` y `ei ∉ C` tal que `ei → ej`. Esto representaría un estado imposible (recibir un mensaje que "aún no fue enviado").

Estado Global Consistente

Se puede definir un **estado global consistente** como aquel que corresponde a un corte consistente. Una secuencia de transiciones de estados globales consistentes en un sistema distribuido determina una **ejecución consistente**.

Un **snapshot** es una instantánea del sistema en un punto consistente, a partir de la cual se podrá recuperar el sistema ante una falla.

Algoritmo de Snapshot de Chandy y Lamport

Chandy y Lamport (1985) propusieron un algoritmo para capturar un estado global consistente **sin detener el sistema**. Supone canales FIFO.

Reglas del algoritmo:

Regla de envío de marcador (marker sending rule): Cuando un proceso `Pi` graba su estado local por primera vez, envía un mensaje especial llamado **marcador** (*marker*) por cada canal de salida, antes de cualquier otro mensaje.

Regla de recepción de marcador (marker receiving rule): Cuando `Pj` recibe un marcador por el canal `Cij`:

- Si `Pj` **aún no grabó su estado**: graba su estado local, graba el estado del canal `Cij` como vacío, y aplica la regla de envío de marcador.
- Si `Pj` **ya grabó su estado**: graba el estado del canal `Cij` como la secuencia de mensajes que recibió por ese canal desde que grabó su estado hasta que recibió el marcador.

Propiedades del algoritmo:

- El estado resultante es un corte **consistente** (puede demostrarse formalmente).
- El algoritmo termina en tiempo finito (si los canales son FIFO y confiables).
- No requiere detener los procesos; el sistema sigue ejecutando durante el snapshot.
- El estado capturado no es necesariamente el estado en un instante real del sistema, pero es un estado que **podría haber ocurrido** y es válido para detectar predicados estables.

Predicados estables: propiedades que, una vez verdaderas, permanecen verdaderas. El deadlock es un predicado estable (si hay deadlock en el momento del snapshot, hay deadlock en la realidad). La terminación también. La ausencia de garbage también.

Exclusión Mutua Distribuida

En sistemas distribuidos, múltiples procesos pueden querer acceder simultáneamente a una **sección crítica**. La regla es:

> Si un proceso está ejecutando en la sección crítica, ningún otro puede hacerlo.

Tres propiedades que debe cumplir cualquier solución:

- **ME1 (Safety)**: como máximo un proceso está en la sección crítica a la vez.
- **ME2 (Liveness)**: toda solicitud de entrar a la sección crítica eventualmente se concede.
- **ME3 (Ordering)**: si una solicitud `a` ocurrió antes que `b` (en el sentido de Lamport), `a` se concede antes.

Algoritmo Centralizado

Un proceso especial actúa como **coordinador**. Cuando un proceso desea entrar a la sección crítica:

1. Envía una solicitud al coordinador indicando la región crítica y solicitando permiso.
2. Si ningún otro proceso tiene el "token", el coordinador responde otorgándolo.

3. Si la región está ocupada, el coordinador **encola** la solicitud sin responder.
4. Cuando el proceso sale, envía un mensaje de liberación al coordinador.
5. El coordinador extrae el primer elemento de la cola y le otorga el permiso.

Costo: 3 mensajes por ciclo (request + grant + release). **Ventajas**: simple, FIFO. **Desventajas**: punto único de falla, cuello de botella. No satisface ME3.

Algoritmo en Anillo (Token Ring)

Los N procesos se organizan por software en un **anillo lógico**. Al inicializar, el proceso 0 recibe un **token** que circula en un único sentido de `pk` a `p(k+1) mod N`.

- Si el proceso recibe el token y **no** desea entrar: lo pasa inmediatamente.
- Si **sí** desea entrar: retiene el token, ejecuta la sección crítica, luego pasa el token.

Ventaja: sin punto único de falla del coordinador. **Desventaja**: el token circula permanentemente (costo de red); si el token se pierde, debe regenerarse.

Algoritmo Distribuido de Ricart y Agrawala (1981)

Cuando un proceso desea entrar a una región crítica:

1. Construye un mensaje con: nombre de la región, su ID de proceso, y su **marca de tiempo** de Lamport.
2. Envía este mensaje a **todos los demás procesos**.
3. Espera `OK` de todos los demás (n-1 respuestas).

Cuando un proceso recibe una solicitud:

- **Si no está en la RC y no quiere entrar**: responde `OK`.
- **Si ya está en la RC**: encola la solicitud.
- **Si también quiere entrar**: compara marcas de tiempo. La **marca más baja gana** (desempate por ID). Si la recibida es menor, responde `OK`; si no, encola.

Cuando un proceso sale de la RC: envía `OK` a todos los procesos en su cola.

Costo: 2(N-1) mensajes por acceso. **Ventaja**: cumple ME1, ME2 y ME3; sin punto único de falla. **Desventaja**: frágil ante fallos (un proceso que no responde bloquea a todos).

Elecciones de Coordinador (Coulouris, Cap. 15.3)

Muchos algoritmos distribuidos requieren un coordinador (ej. el coordinador del algoritmo centralizado de exclusión mutua, el master del algoritmo de Berkeley). El problema de **elecciones** consiste en elegir un proceso coordinador entre un conjunto de procesos cuando el coordinador actual falla.

Propiedades que debe cumplir cualquier algoritmo de elección:

- **E1 (Safety)**: todos los procesos correctos que participen en la elección deben concordar en el mismo proceso elegido.
- **E2 (Liveness)**: todos los procesos correctos eventualmente completan la elección y participan en el resultado.

El proceso elegido es generalmente el que tiene el **mayor identificador** (o la mayor prioridad) entre los procesos activos.

Algoritmo de Elección en Anillo

Similar al anillo de exclusión mutua pero para elecciones. Cuando un proceso `pi` detecta que el coordinador falló:

1. `pi` envía un mensaje de elección con su propio identificador al siguiente proceso en el anillo.
2. Cada proceso que recibe el mensaje: si el ID del mensaje es mayor que el propio, lo reenvía. Si es menor, lo reemplaza por el propio ID y lo reenvía. Si es igual al propio, el proceso se declara coordinador y envía un mensaje de coordinador alrededor del anillo.

El proceso elegido es el de mayor ID entre los activos. El algoritmo tolera cero fallos durante la elección (si un proceso falla durante el proceso, el anillo queda roto).

Algoritmo Bully (Garcia-Molina, 1982)

El algoritmo Bully permite que procesos fallen **durante** la elección. Supone un sistema síncrono (timeouts deterministas) y que cada proceso conoce los identificadores de todos los demás.

Tipos de mensajes:

- `election`: anuncia el inicio de una elección.
- `answer`: respuesta a un mensaje `election`.
- `coordinator`: anuncia el nuevo coordinador.

Funcionamiento:

Un proceso `pi` comienza una elección cuando detecta (por timeout) que el coordinador falló:

1. `pi` envía un mensaje `election` a todos los procesos con identificador mayor.
2. Si ninguno responde en tiempo `T = 2×Ttrans + Tprocess`:
    - `pi` se declara coordinador y envía `coordinator` a todos los procesos con identificador menor.
3. Si algún proceso `pj` (con ID mayor) responde con `answer`:
    - `pi` espera un mensaje `coordinator` de `pj` por tiempo `Tc`.
    - Si no llega, `pi` comienza una nueva elección.
4. Cuando `pi` recibe un mensaje `election` de otro proceso: envía `answer` y comienza su propia elección (si no lo ha hecho).
5. Si `pi` recibe un mensaje `coordinator`, registra al nuevo coordinador y termina.

¿Por qué se llama "bully"? Porque si un proceso con identificador alto se inicia (o se recupera tras un crash), puede "intimidar" (bully) a procesos que ya habían elegido un coordinador de menor ID, reemplazándolo.

Análisis:

- **Mejor caso**: el proceso con el segundo mayor ID detecta el fallo → N-2 mensajes `coordinator` → O(N) mensajes.
- **Peor caso**: el proceso con el menor ID detecta el fallo → múltiples rondas → O(N²) mensajes.
- **Correctitud**: cumple E2 si la entrega de mensajes es confiable. E1 puede romperse si el sistema no es perfectamente síncrono (dos procesos pueden creerse coordinadores simultáneamente).

6.2 Transacciones Distribuidas

Motivación: de la Sincronización de Bajo Nivel a las Transacciones

Las técnicas de sincronización vistas son de **bajo nivel**: el programador debe gestionar directamente la exclusión mutua, las regiones críticas, la prevención de bloqueos y la recuperación ante fallas.

Se necesita una abstracción de **mayor nivel** que oculte estos detalles y permita a los programadores concentrarse en la lógica del negocio. Esa abstracción se llama **transacción atómica**.

Una transacción es **un conjunto de operaciones sobre datos que son tratadas como una unidad indivisible**.

Propiedades ACID

Härder y Reuter (1983) formalizaron las propiedades con el acrónimo **ACID**:

Propiedad	Descripción
Atomicity (Atomicidad)	La transacción es "todo o nada". O se ejecutan todas sus operaciones, o no se ejecuta ninguna.
Consistency (Consistencia)	Una transacción lleva al sistema de un estado consistente a otro estado consistente. Es responsabilidad de los programadores garantizarla.
Isolation (Aislamiento)	Cada transacción se ejecuta sin interferencia de otras. Los efectos intermedios de una transacción no son visibles para otras transacciones concurrentes.
Durability (Durabilidad)	Una vez completada exitosamente (*committed*), los efectos de la transacción se guardan en almacenamiento permanente y sobreviven a fallos del servidor.

La **atomicidad** tiene dos dimensiones:

- **Atomicidad ante falla** (*failure atomicity*): los efectos son atómicos incluso cuando el servidor cae.
- **Durabilidad**: los cambios persisten en disco una vez confirmados.

Serialización y Control de Concurrencia

Si un servidor acepta transacciones concurrentes, debe sincronizar sus operaciones para garantizar el aislamiento. Hay dos enfoques básicos:

- **Transacciones en serie**: ejecutarlas de a una. Correcto pero sin concurrencia.
- **Serialización**: ejecutar varias concurrentemente, garantizando que el resultado sea equivalente a alguna ejecución serial.

En el contexto distribuido, si dos transacciones T y U acceden conflictivamente a un objeto en un servidor, ese orden debe mantenerse consistente **en todos los servidores** que ambas transacciones visiten.

Locking (Two-Phase Locking) — Coulouris, Cap. 16.4

El control de concurrencia por **locks** es el mecanismo más usado en bases de datos relacionales. Cada objeto tiene un lock que debe ser adquirido antes de acceder al objeto y liberado después.

Tipos de locks:

- **Read lock (shared lock)**: múltiples transacciones pueden tener read lock sobre el mismo objeto simultáneamente.
- **Write lock (exclusive lock)**: solo una transacción puede tener write lock sobre un objeto. Impide cualquier otro lock.

Compatibilidad de locks:

	Read lock solicitado	**Write lock solicitado**
Ningún lock	Conceder	Conceder
Read lock vigente	Conceder	Esperar
Write lock vigente	Esperar	Esperar

Two-Phase Locking (2PL): protocolo que garantiza serialización. Una transacción:

1. **Fase de crecimiento (growing phase)**: puede adquirir locks, pero **no puede liberarlos**.
2. **Fase de decrecimiento (shrinking phase)**: puede liberar locks, pero **no puede adquirir nuevos**.

El punto donde la transacción adquiere el último lock se llama **lock point**. El orden serial equivalente está determinado por el orden de los lock points.

Strict 2PL: variante donde los locks se liberan **solo al hacer commit o abort**. Evita lecturas sucias (dirty reads) y es la variante más usada en la práctica.

Problema del locking: bloqueo (blocking). Mientras una transacción espera un lock, sus recursos permanecen bloqueados. Si la espera es larga (porque otra transacción tiene el lock por mucho tiempo), el rendimiento se degrada.

Control Optimista de la Concurrencia

En lugar de usar locks, este esquema asume que los conflictos son raros. Tiene tres fases:

1. **Fase de trabajo**: cada transacción trabaja sobre **versiones tentativas** privadas de los objetos. Los valores tentatives son invisibles para otras transacciones. Se registran un **conjunto de lectura** y un **conjunto de escritura** por cada transacción.

2. **Fase de validación**: cuando se solicita `closeTransaction`, se valida si las operaciones de la transacción entran en conflicto con las de otras transacciones sobre los mismos objetos.

    - Si la validación es exitosa → commit.
    - Si falla → abort (puede haber cascadas de abort).
3. **Fase de actualización**: si la transacción se valida, todos los cambios de sus versiones tentativas se vuelven permanentes.

Timestamp Ordering — Coulouris, Cap. 16.6

El control de concurrencia por **timestamps** asigna a cada transacción un timestamp único cuando comienza. El sistema garantiza que el orden de ejecución es equivalente al orden serial determinado por los timestamps.

Reglas para operaciones de lectura/escritura:

- Cuando `Ti` intenta leer el objeto `O`: si `Ti` tiene un timestamp mayor al timestamp de la última escritura confirmada de `O`, la operación procede. Si el timestamp de `Ti` es menor, `Ti` está intentando leer un valor "del futuro" que ya fue sobreescrito → **abort de Ti**.
- Cuando `Ti` intenta escribir el objeto `O`: si existe una transacción `Tj` con mayor timestamp que ya leyó el valor que `Ti` quiere sobreescribir → **abort de Ti** (no puede sobreescribir una lectura futura ya ocurrida).

En transacciones distribuidas: se requieren timestamps **globalmente únicos**. Esto se logra combinando el timestamp local con el ID del servidor: `<timestamp_local, server_id>`.

Comparación de métodos:

Método	Conflictos	Bloqueo	Escalabilidad	Adecuado para
Locking (2PL)	Espera	Sí (puede haber deadlock)	Media	Muchos conflictos
Optimista	Abort y retry	No	Alta	Pocos conflictos
Timestamp Ordering	Abort y retry	No	Media-alta	Patrones predecibles

---

Implementación: Métodos para Garantizar Atomicidad

Espacio de Trabajo Privado

Al iniciarse una transacción, se le otorga al proceso un **espacio de memoria particular** con copias de todos los archivos a los que tiene acceso. Todas las lecturas y escrituras van a este espacio privado. Solo si la transacción hace *commit* los cambios se vuelven visibles al mundo exterior.

- **Ventaja**: aumenta el grado de concurrencia (los demás no ven cambios intermedios).
- **Desventaja**: costo elevado por el copiado inicial al espacio de trabajo.

Log de Escritura Adelantada (Write-Ahead Log)

Antes de escribir cualquier bloque de datos, se escribe primero un **registro en el log del sistema** (almacenamiento estable) que contiene:

- La transacción que realizó el cambio.
- El archivo o bloque modificado.
- Los valores **anteriores** y **nuevos** del dato.

Solo después de escribir en el log se realizan los cambios en el archivo real.

- Si la transacción tiene éxito y hace commit: se registra el commit en el log.
- Si la transacción se aborta: se usa el log para **deshacer** los cambios (rollback).

Ejemplo:

X = 0, Y = 0

Begin transaction

x = x + 1 → Log: X = 0/1

y = y + 2 → Log: Y = 0/2

y = y * y → Log: Y = 2/4

End transaction

Si se aborta, el log permite deshacer exactamente en orden inverso.

Transacciones Distribuidas: El Problema del Commit

En un sistema distribuido, una transacción puede involucrar **múltiples servidores** en distintos nodos. El desafío es que el resultado (commit o abort) debe ser **el mismo en todos los participantes**.

Protocolo One Phase Commit (1PC)

El coordinador comunica el requerimiento a todos los participantes y lo repite hasta que todos confirmen.

Problema crítico: si hay fallas en la red, la transacción queda bloqueada. Los participantes no pueden decidir unilateralmente (commit sin autorización → incorrecto si el coordinador decidió abort; abort → incorrecto si el coordinador decidió commit). Por eso el 1PC es **insuficiente** para transacciones distribuidas.

Protocolo Two Phase Commit (2PC)

El 2PC es el protocolo estándar para el commit atómico distribuido. Separa el proceso en dos fases: **votación** y **completación**.

Operaciones del protocolo (Coulouris, Fig. 17.4):

Operación	Quién la llama	Destino	Descripción
`canCommit?(trans)`	Coordinador	Participante	Pregunta si está preparado para commit. Responde Yes/No.
`doCommit(trans)`	Coordinador	Participante	Ordena hacer commit.
`doAbort(trans)`	Coordinador	Participante	Ordena hacer abort.
`haveCommitted(trans, participant)`	Participante	Coordinador	Confirma que completó el commit.
`getDecision(trans)`	Participante	Coordinador	Consulta la decisión cuando no recibió respuesta tras timeout.

Fase 1 — Votación

1. El coordinador escribe "prepare to commit" en su log y envía `canCommit?` a todos los participantes.
2. Cada participante: si puede commit, escribe "agree to commit" en su log y responde `Yes`. Si no puede, responde `No` y aborta inmediatamente.

Fase 2 — Completación

3. El coordinador recolecta los votos:
   - (a) Si **todos** votaron `Yes`: decide commit, envía `doCommit` a todos.
   - (b) Si **alguno** votó `No` o hubo timeout: decide abort, envía `doAbort` a los que votaron `Yes`.
4. Los participantes que votaron `Yes` esperan `doCommit` o `doAbort`. Al recibirlo, actúan en consecuencia. En caso de commit, envían `haveCommitted` al coordinador.

Costo normal (sin fallas, N participantes): N mensajes `canCommit` + N respuestas + N mensajes `doCommit` = **3N mensajes**, equivalente a 3 rondas.

Manejo de Timeouts y Fallos

Participante en estado incierto (ya votó `Yes`, esperando decisión): no puede decidir unilateralmente. Intenta `getDecision` al coordinador. Si el coordinador falló, **queda bloqueado** hasta que el coordinador sea reemplazado.

El 2PC garantiza completar eventualmente, pero sin límite de tiempo definido.

2PC para Transacciones Anidadas

Las transacciones pueden ser **anidadas**: una transacción padre puede lanzar subtransacciones hijas. La transacción más externa es la **top-level transaction**.

Hay dos variantes:

Protocolo 2PC Jerárquico

El coordinador del top-level envía `canCommit?(trans, subTrans)` a sus hijas directas. Cada una reenvía el `canCommit?` a sus propias hijas recursivamente. Cada participante recolecta respuestas de sus descendientes antes de responder a su padre.

Protocolo 2PC Plano (Flat)

El coordinador del top-level envía `canCommit?(trans, abortList)` directamente a **todos** los coordinadores de subtransacciones en la lista de commit provisional. El parámetro `abortList` lista las subtransacciones que abortaron, para que los participantes no commiteen descendientes con ancestros abortados.

Deadlocks Distribuidos (Coulouris, Cap. 17.5)

En el control de concurrencia por locking, el deadlock ocurre cuando dos o más transacciones se esperan mutuamente en un ciclo:

T espera lock de A (que tiene U)

U espera lock de B (que tiene T)

→ Deadlock: ciclo T → U → T

En un sistema distribuido, el deadlock puede involucrar transacciones en **múltiples servidores**. Cada servidor solo ve su porción del grafo de espera (wait-for graph) local. El ciclo puede no ser visible en ningún servidor individual.

Ejemplo de deadlock distribuido:

Servidor X: T tiene lock de A, U espera A

Servidor Y: U tiene lock de B, T espera B

Servidor X ve: U → T. Servidor Y ve: T → U. Ninguno ve el ciclo completo. El sistema está deadlocked pero ningún servidor puede detectarlo localmente.

Detección de Deadlocks Distribuidos

La detección requiere construir el **grafo de espera global**. El enfoque estándar es el de un **detector de deadlock centralizado**:

1. Periódicamente (o cuando se sospecha un deadlock), cada servidor envía su grafo de espera local a un detector centralizado.
2. El detector construye la unión de todos los grafos locales.
3. El detector busca ciclos en el grafo global.
4. Si se detecta un ciclo, se selecciona una transacción víctima para abortar (típicamente la más joven o la de menor prioridad).

Problema del detector centralizado: puede detectar deadlocks que no existen realmente (*phantom deadlocks*) si los mensajes de diferentes servidores llegan en momentos distintos y reflejan estados inconsistentes del tiempo.

Alternativas:

- **Detección por timeouts**: si una transacción espera más de un umbral de tiempo, se asume deadlock y se aborta. Simple pero impreciso.
- **Algoritmos distribuidos de detección** (Chandy-Misra-Haas): cada proceso propagates información de espera hacia sus vecinos. Un ciclo se detecta cuando un mensaje de espera regresa al proceso que lo originó.

Resolución de Deadlocks

Una vez detectado un ciclo, se selecciona una transacción **víctima** para abortar. El coordinador informa a los participantes involucrados en esa transacción para que la aborten, liberando los locks y rompiendo el ciclo.

Recuperación de Transacciones (Coulouris, Cap. 17.6)

La recuperación garantiza que el sistema pueda volver a un estado consistente después de un fallo (crash de un servidor, pérdida de mensajes, corte de red).

Tipos de fallo y sus efectos

Tipo de fallo	Qué puede perderse
Crash de proceso	Estado volátil (memoria RAM)
Crash de disco	Datos en almacenamiento

Corte de red	Mensajes en tránsito

Principios de recuperación

Almacenamiento estable (stable storage): almacenamiento que sobrevive a fallos del servidor. Se logra mediante:

- Replicación del disco (RAID).
- Escritura en dos discos físicos independientes antes de confirmar.

Write-ahead logging (WAL): el log debe escribirse en almacenamiento estable **antes** de cualquier modificación al objeto. Esto garantiza que el log siempre refleja lo que ocurrió, incluso si el servidor cae.

Procedimiento de recuperación tras un crash

Cuando un servidor se reinicia después de un crash, ejecuta un procedimiento de recuperación sobre el log:

1. **Para transacciones committed** (el log contiene un registro de commit): se aplican **REDO** (rehacer las operaciones). Asegura durabilidad.
2. **Para transacciones abortadas o incompletas** (el log no contiene commit): se aplican **UNDO** (deshacer las operaciones). Asegura atomicidad.

Recuperación en el contexto del 2PC

Si un **participante** cae durante el 2PC:

- Si cayó antes de responder `canCommit?`: al recuperarse, puede abortar la transacción (nunca prometió commit).
- Si cayó después de responder `Yes` (en estado "prepared"): al recuperarse, **no puede decidir unilateralmente**. Debe contactar al coordinador con `getDecision` para saber si commitear o abortar. Mientras tanto, sus locks permanecen retenidos.

Si el **coordinador** cae:

- Los participantes que votaron `Yes` quedan en estado incierto indefinidamente hasta que el coordinador se recupere.
- Al recuperarse, el coordinador lee su log para determinar en qué fase estaba y retoma el protocolo desde ese punto.

Checkpoints: para reducir el tiempo de recuperación, los servidores periódicamente escriben un **checkpoint** al log, marcando que todos los cambios anteriores están garantizados en disco. Así, la recuperación solo necesita procesar el log desde el último checkpoint.

Resumen Comparativo de Protocolos de Commit

Aspecto	One Phase Commit	Two Phase Commit

Fases	1	2
Bloqueo ante falla del coordinador	Sí, sin solución segura	Sí, pero con `getDecision`
Decisión unilateral del participante	Posible pero incorrecta	No permitida
Costo en mensajes	2N	3N
Uso práctico	No usado	Estándar de la industria

Conexión entre 6.1 y 6.2

La sincronización (6.1) y las transacciones (6.2) no son temas independientes: las transacciones son la **abstracción de alto nivel** que se construye *sobre* los mecanismos de sincronización de bajo nivel.

El propio libro de Coulouris lo señala explícitamente: las técnicas de exclusión mutua y manejo de regiones críticas son de bajo nivel; las transacciones proveen la abstracción que permite al programador ignorar esos detalles. El 2PC usa un protocolo de acuerdo distribuido que depende de comunicación punto a punto, timeouts y recuperación ante fallos — todo lo estudiado en sincronización de bajo nivel.

El snapshot de Chandy-Lamport, a su vez, conecta ambos mundos: se basa en los relojes vectoriales y la relación "ocurre antes" (6.1) para capturar estados globales consistentes que permiten detectar deadlocks distribuidos (6.2).

Fuentes: Sincronizacion.pdf, Transacciones.pdf, split-document_colouris.pdf (Cap. 14, 15, 16, 17)